Technical Task

Situation

Hello, our future Data Engineer. The following exercise has been designed to see how you turn messy, real-world public data into something people can trust, and how you make and defend tradeoffs when the data doesn't cooperate. Use coding agents & tools (e.g. Claude Code, Cursor or any other) however you want — we do.

Resources Provided

Public data sources you may find useful (starting points, not requirements)

- Lithuanian Open Data Portal — data.gov.lt (including its API at get.data.gov.lt)
- Sodra open company data — atvira.sodra.lt
- Registrų centras — registrucentras.lt

You may also use Oxylabs (or any other proxy/scraper provider) products if needed, most of them offer limited free trials.

Heads up before you start

This is a deliberately open task, sized for roughly five hours of work with AI tools. **We do not expect a production-ready end-to-end system.** Focus on demonstrating data modelling, data quality and system design thinking. Pick what you think matters most, build that well, and be explicit about what you cut and why. A sharp, well-reasoned slice beats a sprawling half-built everything.

Your task

The task will require you to **build a system that surfaces shadow-economy signals among Vilnius businesses** — places that look busy to the public, but quiet on paper.

Your audience is an analyst at the State Tax Inspectorate (VMI) with limited inspection capacity, who needs to decide where to look first. Think of the output as a *prioritised lead list*: a café with 2,000 Google reviews and a weekend queue that declares one employee is worth a closer look; a quiet corner shop with one employee is not.

How you collect the data, how you link a public listing to a legal entity, what you store, how you define "busy" and "quiet", how you score and explain a lead, how you prove it works — all yours to design.

This is a hypothetical scenario created for this exercise only.

Expectations & ground rules

Data Collection

Visible business activity should come from **Google** (e.g. Google Maps / local results), collected using **Python**. Declared activity should come from **official Lithuanian public sources**. Cover at least **3 business categories** in Vilnius of your choice. Respect ethical scraping practices and each source's terms of use.

System Implementation

The system should produce a real lead list from real, recent data. Every lead should be explainable and traceable — the analyst must be able to see which listing and which official records it is based on, and why it scored the way it did.

The analyst should be able to ask their own follow-up questions of your data (e.g. "which categories have the highest share of flagged businesses?") without re-running the pipeline.

The analyst plans to run this every month and compare results over time.

The analyst has to be able to trust the list. These are leads, not accusations — think about the cost of flagging an honest business and reflect it in your design.

That's it. Everything else — architecture, storage, matching, scoring, error handling, tool & framework selection — is your call.

Containerization & Version Control

Provide your solution in a containerized format — we should be able to run it with a single command on a clean machine. Host your answers in a public **GitHub repository with clear documentation**. We will read the commit history.

Commit an example output generated from a real run, and a short **DECISIONS.md** (1–2 pages): your key design decisions, the alternatives you rejected, and known limitations.

(Later) Presentation (~20 mins of the next interview)

1. **Showcase** your system (running the application *locally*).
2. **Present limitations** of your current implementation and areas where it might fail — especially wrong matches and false positives.
3. (Important) **Propose improvements** for a production-ready version, considering aspects like:
    - Scalability — from 3 categories in Vilnius to every business in Lithuania;
    - Accuracy of matching and scoring;
    - Data freshness and update mechanisms (registers and listings change at different rates);
    - Monitoring, observability and data quality checks;
    - Cost optimization.

Repo walk-through, live solution demo and a README that covers all points is sufficient in the presentation part.

Good luck!